

Análise Sintática



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismailly
ismailybf@ufc.br

Semestre 2025.2

Linguagens de Programação
Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)

Seção 1

Revisão

Seção 2

LR Parsing

LR Parsing

- O ponto fraco da técnica LL(k) é precisar prever que produção usar

LR Parsing

- O ponto fraco da técnica LL(k) é precisar prever que produção usar
 - Com base nos primeiros k tokens do lado direito da produção.

LR Parsing

- O ponto fraco da técnica LL(k) é precisar prever que produção usar
 - Com base nos primeiros k tokens do lado direito da produção.
- LR(k) posterga a decisão até ter visto todo o lado direito de uma produção, mais os k próximos tokens da entrada.

LR Parsing

- O ponto fraco da técnica LL(k) é precisar prever que produção usar
 - Com base nos primeiros k tokens do lado direito da produção.
- LR(k) posterga a decisão até ter visto todo o lado direito de uma produção, mais os k próximos tokens da entrada.
 - *Left-to-right parse, rightmost-derivation, k-token lookahead.*

LR Parsing

- O parser tem uma pilha e a entrada.

LR Parsing

- O parser tem uma pilha e a entrada.
- Os primeiros k tokens da entrada forma o lookahead.

LR Parsing

- O parser tem uma pilha e a entrada.
- Os primeiros k tokens da entrada forma o lookahead.
- Dois tipos de ações:

LR Parsing

- O parser tem uma pilha e a entrada.
- Os primeiros k tokens da entrada forma o lookahead.
- Dois tipos de ações:
 - SHIFT: move o primeiro token para o topo da pila.

LR Parsing

- O parser tem uma pilha e a entrada.
- Os primeiros k tokens da entrada forma o lookahead.
- Dois tipos de ações:
 - SHIFT: move o primeiro token para o topo da pila.
 - REDUCE:

LR Parsing

- O parser tem uma pilha e a entrada.
- Os primeiros k tokens da entrada forma o lookahead.
- Dois tipos de ações:
 - SHIFT: move o primeiro token para o topo da pila.
 - REDUCE:
 - escolhe uma produção $X \rightarrow A B C$;

LR Parsing

- O parser tem uma pilha e a entrada.
- Os primeiros k tokens da entrada forma o lookahead.
- Dois tipos de ações:
 - SHIFT: move o primeiro token para o topo da pila.
 - REDUCE:
 - escolhe uma produção $X \rightarrow A B C$;
 - desempilha C , B e A .

LR Parsing

- O parser tem uma pilha e a entrada.
- Os primeiros k tokens da entrada forma o lookahead.
- Dois tipos de ações:
 - SHIFT: move o primeiro token para o topo da pila.
 - REDUCE:
 - escolhe uma produção $X \rightarrow A B C$;
 - desempilha C , B e A .
 - empilha X .

LR Parsing

- 0. $S' \longrightarrow S\$;$
- 1. $S \longrightarrow S;S$
- 2. $S \longrightarrow \text{id} := E$
- 3. $S \longrightarrow \text{print}(L)$
- 4. $E \longrightarrow \text{id}$
- 5. $E \longrightarrow \text{num}$

- 6. $E \longrightarrow E + E$
- 7. $E \longrightarrow (S,E)$
- 8. $L \longrightarrow E$
- 9. $L \longrightarrow L, E$

LR Parsing

- 0. $S' \longrightarrow S\$;$
- 1. $S \longrightarrow S;S$
- 2. $S \longrightarrow \text{id} := E$
- 3. $S \longrightarrow \text{print}(L)$
- 4. $E \longrightarrow \text{id}$
- 5. $E \longrightarrow \text{num}$

- 6. $E \longrightarrow E + E$
- 7. $E \longrightarrow (S,E)$
- 8. $L \longrightarrow E$
- 9. $L \longrightarrow L, E$

Derivação para:

$a := 7;$

$b := c + (d := 5+6, d);$

Exemplo

Stack	Input	Action
1	a := 7 ; b := c + (d := 5 + 6 , d) \$	shift
1 id ₄	:= 7 ; b := c + (d := 5 + 6 , d) \$	shift
1 id ₄ := ₆	7 ; b := c + (d := 5 + 6 , d) \$	shift
1 id ₄ := ₆ num ₁₀	; b := c + (d := 5 + 6 , d) \$	reduce $E \rightarrow \text{num}$
1 id ₄ := ₆ E ₁₁	; b := c + (d := 5 + 6 , d) \$	reduce $S \rightarrow \text{id} := E$
1 S ₂	; b := c + (d := 5 + 6 , d) \$	shift
1 S ₂ ;	b := c + (d := 5 + 6 , d) \$	shift
1 S ₂ ; id ₄	: = c + (d := 5 + 6 , d) \$	shift
1 S ₂ ; id ₄ := ₆	c + (d := 5 + 6 , d) \$	shift
1 S ₂ ; id ₄ := ₆ id ₂₀	+ (d := 5 + 6 , d) \$	reduce $E \rightarrow \text{id}$
1 S ₂ ; id ₄ := ₆ E ₁₁	+ (d := 5 + 6 , d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆	(d := 5 + 6 , d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8	d := 5 + 6 , d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 id ₄	: = 5 + 6 , d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 id ₄ := ₆	5 + 6 , d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 id ₄ := ₆ num ₁₀	+ 6 , d) \$	reduce $E \rightarrow \text{num}$
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 id ₄ := ₆ E ₁₁	+ 6 , d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 id ₄ := ₆ E ₁₁ + ₁₆	6 , d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 id ₄ := ₆ E ₁₁ + ₁₆ num ₁₀	, d) \$	reduce $E \rightarrow \text{num}$
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 id ₄ := ₆ E ₁₁ + ₁₆ E ₁₇	, d) \$	reduce $E \rightarrow E + E$
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 id ₄ := ₆ E ₁₁	, d) \$	reduce $S \rightarrow \text{id} := E$
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 S ₁₂	, d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 S ₁₂ ,18	d) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 S ₁₂ ,18 id ₂₀	) \$	reduce $E \rightarrow \text{id}$
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 S ₁₂ ,18 E ₂₁	) \$	shift
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ (8 S ₁₂ ,18 E ₂₁) ₂₂	\$	reduce $E \rightarrow (S, E)$
1 S ₂ ; id ₄ := ₆ E ₁₁ + ₁₆ E ₁₇	\$	reduce $E \rightarrow E + E$
1 S ₂ ; id ₄ := ₆ E ₁₁	\$	reduce $S \rightarrow \text{id} := E$
1 S ₂ ; S ₅	\$	reduce $S \rightarrow S; S$
1 S ₂	\$	accept

LR Parsing Engine

- Como o parser sabe quando fazer um SHIFT ou um REDUCE?

LR Parsing Engine

- Como o parser sabe quando fazer um SHIFT ou um REDUCE?
- Usando um DFA aplicado a pilha!!

LR Parsing Engine

- Como o parser sabe quando fazer um SHIFT ou um REDUCE?
- Usando um DFA aplicado a pilha!!
- As arestas são nomeadas com os símbolos que podem aparecer na pilha.

Exemplo

	id	num	print	;	,	+	:=	(	)	\$	S	E	L
1	s4		s7								g2		
2				s3						a			
3	s4		s7								g5		
4							s6						
5				r1	r1					r1			
6	s20	s10						s8				g11	
7								s9					
8	s4		s7								g12		
9	s20	s10						s8				g15	g14
10				r5	r5	r5			r5	r5			
11				r2	r2	s16				r2			
12				s3	s18								
13				r3	r3					r3			
14					s19				s13				
15					r8				r8				
16	s20	s10						s8				g17	
17				r6	r6	s16			r6	r6			
18	s20	s10						s8				g21	
19	s20	s10						s8				g23	
20				r4	r4	r4			r4	r4			
21									s22				
22				r7	r7	r7			r7	r7			
23					r9	s16			r9				

Tabela de transição

- 4 tipos de ações:

Tabela de transição

- 4 tipos de ações:
 - sn: SHIFT para o estado n;

Tabela de transição

- 4 tipos de ações:
 - sn: SHIFT para o estado n;
 - gn: vá para o estado n;

Tabela de transição

- 4 tipos de ações:
 - sn: SHIFT para o estado n;
 - gn: vá para o estado n;
 - rk: REDUCE pela regra k;

Tabela de transição

- 4 tipos de ações:
 - sn: SHIFT para o estado n;
 - gn: vá para o estado n;
 - rk: REDUCE pela regra k;
 - a: Accept;

Tabela de transição

- 4 tipos de ações:
 - sn: SHIFT para o estado n;
 - gn: vá para o estado n;
 - rk: REDUCE pela regra k;
 - a: Accept;
 - : Error (entrada em branco)

Tabela de transição

- 4 tipos de ações:
 - sn: SHIFT para o estado n;
 - gn: vá para o estado n;
 - rk: REDUCE pela regra k;
 - a: Accept;
 - : Error (entrada em branco)
- As arestas do DFA são as ações SHIFT e goto.

Tabela de transição

- 4 tipos de ações:
 - sn: SHIFT para o estado n;
 - gn: vá para o estado n;
 - rk: REDUCE pela regra k;
 - a: Accept;
 - : Error (entrada em branco)
- As arestas do DFA são as ações SHIFT e goto.
- No exemplo anterior, cada número indica o estado destino.

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):
 - A entrada avança um símbolo e é feito um *push* de n na pilha.

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):
 - A entrada avança um símbolo e é feito um *push* de n na pilha.
- Se a ação for REDUCE(k):

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):
 - A entrada avança um símbolo e é feito um *push* de n na pilha.
- Se a ação for REDUCE(k):
 - Desempilhe r símbolos da pilha (*pop*), onde r é o número de símbolos no lado direito da regra k .

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):
 - A entrada avança um símbolo e é feito um *push* de n na pilha.
- Se a ação for REDUCE(k):
 - Desempilhe r símbolos da pilha (*pop*), onde r é o número de símbolos no lado direito da regra k .
 - Seja X o símbolo do lado esquerdo da regra k ;

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):
 - A entrada avança um símbolo e é feito um *push* de n na pilha.
- Se a ação for REDUCE(k):
 - Desempilhe r símbolos da pilha (*pop*), onde r é o número de símbolos no lado direito da regra k .
 - Seja X o símbolo do lado esquerdo da regra k ;
 - No estado (atual) do topo da pilha, procure X para obter um “goto n ”.

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):
 - A entrada avança um símbolo e é feito um *push* de n na pilha.
- Se a ação for REDUCE(k):
 - Desempilhe r símbolos da pilha (*pop*), onde r é o número de símbolos no lado direito da regra k .
 - Seja X o símbolo do lado esquerdo da regra k ;
 - No estado (atual) do topo da pilha, procure X para obter um “goto n ”.
 - Adicione X ao topo da pilha.

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):
 - A entrada avança um símbolo e é feito um *push* de n na pilha.
- Se a ação for REDUCE(k):
 - Desempilhe r símbolos da pilha (*pop*), onde r é o número de símbolos no lado direito da regra k .
 - Seja X o símbolo do lado esquerdo da regra k ;
 - No estado (atual) do topo da pilha, procure X para obter um “goto n ”.
 - Adicione X ao topo da pilha.
- Accept: para o parsing e reporta sucesso.

Algoritmo

- Olhe para o estado no topo da pilha e para o símbolo de entrada para determinar a ação.
- Se a ação for SHIFT(n):
 - A entrada avança um símbolo e é feito um *push* de n na pilha.
- Se a ação for REDUCE(k):
 - Desempilhe r símbolos da pilha (*pop*), onde r é o número de símbolos no lado direito da regra k .
 - Seja X o símbolo do lado esquerdo da regra k ;
 - No estado (atual) do topo da pilha, procure X para obter um “goto n ”.
 - Adicione X ao topo da pilha.
- Accept: para o parsing e reporta sucesso.
- Error: para o parsing e reporta falha.

Tabela de transição

- O exemplo anterior mostrou o uso de 1 símbolo de lookahead.

Tabela de transição

- O exemplo anterior mostrou o uso de 1 símbolo de lookahead.
- Para k , a tabela terá colunas para todas as sequências de k tokens.

Tabela de transição

- O exemplo anterior mostrou o uso de 1 símbolo de lookahead.
- Para k , a tabela terá colunas para todas as sequências de k tokens.
- $k > 1$ praticamente não é usado para compilação.

Tabela de transição

- O exemplo anterior mostrou o uso de 1 símbolo de lookahead.
- Para k , a tabela terá colunas para todas as sequências de k tokens.
- $k > 1$ praticamente não é usado para compilação.
- Maioria das linguagens de programação podem ser descritas por gramáticas LR(1).

Geração de parsers LR(0)

LR(0) são as gramáticas que podem ser analisadas olhando somente a pilha.

Geração de parsers LR(0)

LR(0) são as gramáticas que podem ser analisadas olhando somente a pilha.

$$0. S' \longrightarrow S\$;$$

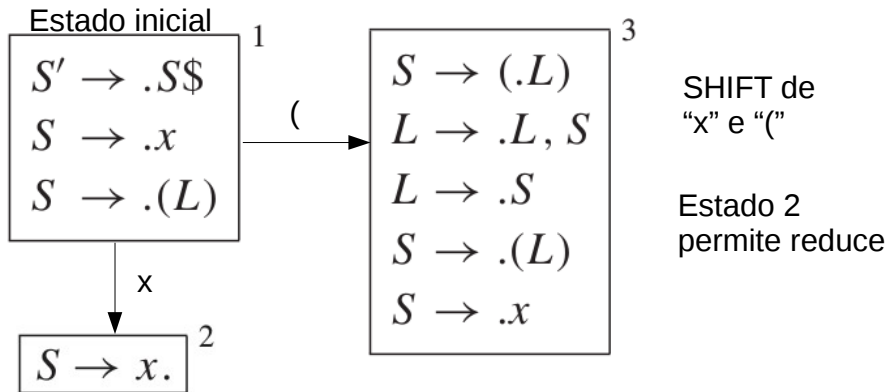
$$1. S \longrightarrow (L)$$

$$2. S \longrightarrow x$$

$$3. L \longrightarrow S$$

$$4. L \longrightarrow L, S$$

Estados



Estados

- Ação de Goto:

Estados

- Ação de Goto:
 - Imagine um SHIFT de “x” ou “(” no estado 1 seguido de redução pela produção de S correspondente.

Estados

- Ação de Goto:
 - Imagine um SHIFT de “x” ou “(” no estado 1 seguido de redução pela produção de S correspondente.
 - Todos os símbolos do lado direito da produção serão desempilhados e o parser vai executar um goto para S no estado 1.

Estados

- Ação de Goto:
 - Imagine um SHIFT de “x” ou “(” no estado 1 seguido de redução pela produção de S correspondente.
 - Todos os símbolos do lado direito da produção serão desempilhados e o parser vai executar um goto para S no estado 1.
 - Isso se representa movendo-se o ponto para após o S e colocando este item em um novo estado (4).

Estados

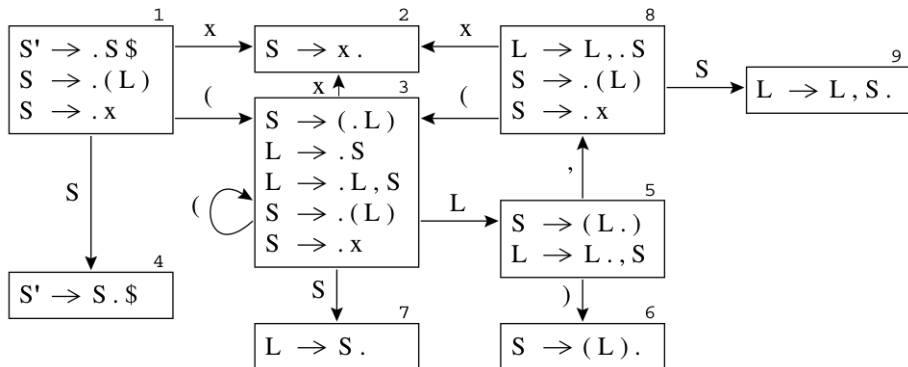
- Ação de Goto:
 - Imagine um SHIFT de “x” ou “(” no estado 1 seguido de redução pela produção de S correspondente.
 - Todos os símbolos do lado direito da produção serão desempilhados e o parser vai executar um goto para S no estado 1.
 - Isso se representa movendo-se o ponto para após o S e colocando este item em um novo estado (4).

Estados

- Ação de Goto:
 - Imagine um SHIFT de “x” ou “(” no estado 1 seguido de redução pela produção de S correspondente.
 - Todos os símbolos do lado direito da produção serão desempilhados e o parser vai executar um goto para S no estado 1.
 - Isso se representa movendo-se o ponto para após o S e colocando este item em um novo estado (4).

$$\boxed{S' \rightarrow S.\4$

Exemplo



Algoritmos

- $\text{Closure}(I)$: adiciona itens a um estado quando um "." precede um não terminal.
- $\text{Goto}(I, X)$: movimenta o "." para depois de X em todos os itens.

Closure(I) =

repeat

for any item $A \rightarrow \alpha.X\beta$ in I

for any production $X \rightarrow \gamma$

$I \leftarrow I \cup \{X \rightarrow .\gamma\}$

until I does not change.

return I

Goto(I, X) =

set J to the empty set

for any item $A \rightarrow \alpha.X\beta$ in I

add $A \rightarrow \alpha.X.\beta$ to J

return $\text{Closure}(J)$

Algoritmos

- Construção do parser LR(0)

Initialize T to $\{\mathbf{Closure}(\{S' \rightarrow .S\})\}$

Initialize E to empty.

repeat

for each state I in T

for each item $A \rightarrow \alpha.X\beta$ in I

let J be **Goto**(I, X)

$T \leftarrow T \cup \{J\}$

$E \leftarrow E \cup \{I \xrightarrow{X} J\}$

until E and T did not change in this iteration

Exemplo

	(	)	x	,	\$	<i>S</i>	<i>L</i>
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

Seção 3

SLR

SLR Parser

$$0. S \rightarrow E\$;$$

$$1. E \rightarrow T + E$$

$$2. E \rightarrow T$$

$$3. T \rightarrow x$$

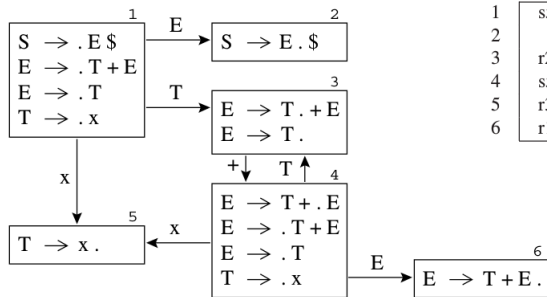
SLR Parser

0. $S \rightarrow E\$;$

1. $E \rightarrow T + E$

2. $E \rightarrow T$

3. $T \rightarrow x$



	x	+	\$	E	T
1	s5			g2	g3
2			a		
3	r2	s4,r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		

SLR Parser

- Colocar reduções somente onde indicado pelo conjunto FOLLOW.

SLR Parser

- Colocar reduções somente onde indicado pelo conjunto FOLLOW.
- Ex.: $\text{FOLLOW}(E) = \{\$\}$

SLR Parser

- Colocar reduções somente onde indicado pelo conjunto FOLLOW.
- Ex.: $\text{FOLLOW}(E) = \{\$, \}$

	x	+	\$	E	T
1	s5			g2	g3
2			a		
3		s4	r2		
4	s5			g6	g3
5		r3	r3		
6			r1		

LR(1)

- Mais poderoso do que SLR.

LR(1)

- Mais poderoso do que SLR.
- Maioria das linguagens de programação são LR(1).

LR(1)

- Mais poderoso do que SLR.
- Maioria das linguagens de programação são LR(1).
 - Exceção notável: C++!!

LR(1)

- Mais poderoso do que SLR.
- Maioria das linguagens de programação são LR(1).
 - Exceção notável: C++!!
- Algoritmo similar ao LR(0).

LR(1)

- Mais poderoso do que SLR.
- Maioria das linguagens de programação são LR(1).
 - Exceção notável: C++!!
- Algoritmo similar ao LR(0).
- Item: $(A \rightarrow \alpha.\beta, x)$.

Algoritmos - Closure(I) e Goto(I,X)

Closure(I) =

repeat

for any item $(A \rightarrow \alpha.X\beta, z)$ in I

for any production $X \rightarrow \gamma$

for any $w \in \text{FIRST}(\beta z)$

$I \leftarrow I \cup \{(X \rightarrow .\gamma, w)\}$

until I does not change

return I

Goto(I, X) =

$J \leftarrow \{\}$

for any item $(A \rightarrow \alpha.X\beta, z)$ in I

add $(A \rightarrow \alpha X.\beta, z)$ to J

return **Closure(J).**

Exemplo

$$0. S \longrightarrow S\$;$$

$$1. S \longrightarrow V = E$$

$$2. S \longrightarrow E$$

$$3. E \longrightarrow V$$

$$4. V \longrightarrow x$$

$$5. V \longrightarrow * E$$

É SLR???

Exemplo

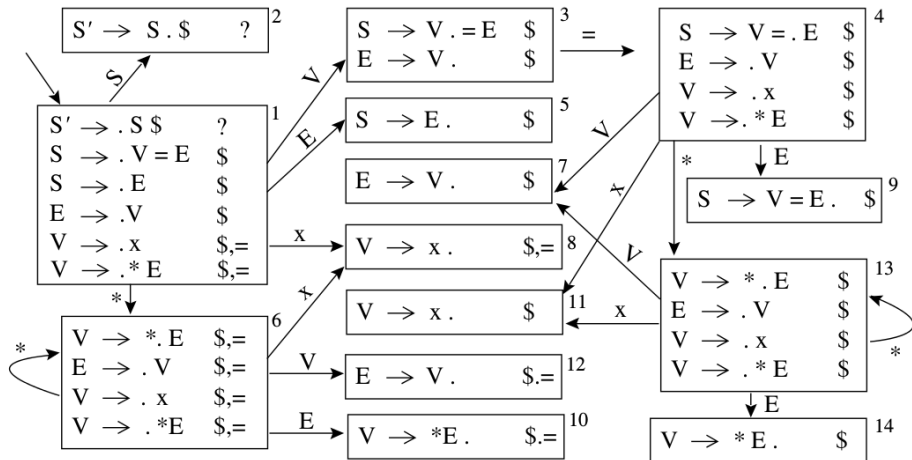
0. $S \longrightarrow S\$;$
1. $S \longrightarrow V = E$
2. $S \longrightarrow E$

3. $E \longrightarrow V$
4. $V \longrightarrow x$
5. $V \longrightarrow * E$

$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$E \rightarrow . V$	\$
$V \rightarrow . x$	\$
$V \rightarrow . * E$	\$
$V \rightarrow . x$	=
$V \rightarrow . * E$	=

$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$E \rightarrow . V$	\$
$V \rightarrow . x$	\$, =
$V \rightarrow . * E$	\$, =

Exemplo



Exemplo

	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s11	s13				g9	g7
5				r2			
6	s8	s6				g10	g12
7				r3			
8			r4	r4			
9				r1			
10			r5	r5			
11				r4			
12			r3	r3			
13	s11	s13				g14	g7
14				r5			

$$R \leftarrow \{\}$$

for each state I in T

for each item $(A \rightarrow \alpha. , z)$ in I

$$R \leftarrow R \cup \{(I, z, A \rightarrow \alpha)\}$$

Parser LALR(1)

- O tamanho das tabelas LR(1) pode ser muito grande.

Parser LALR(1)

- O tamanho das tabelas LR(1) pode ser muito grande.
- É possível reduzir o tamanho unindo estados do DFA.

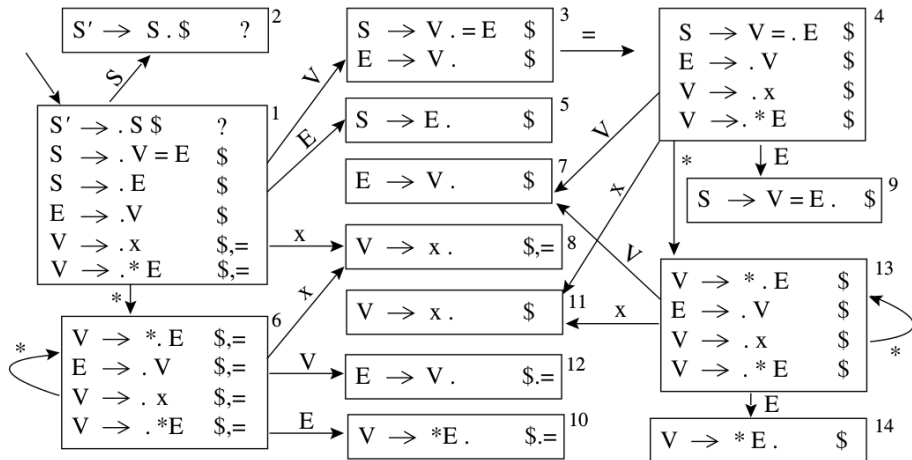
Parser LALR(1)

- O tamanho das tabelas LR(1) pode ser muito grande.
- É possível reduzir o tamanho unindo estados do DFA.
 - Junte os estados que possuam os itens idênticos, exceto pelo lookahead.

Parser LALR(1)

- O tamanho das tabelas LR(1) pode ser muito grande.
- É possível reduzir o tamanho unindo estados do DFA.
 - Junte os estados que possuam os itens idênticos, exceto pelo lookahead.
- Vejamos o exemplo anterior.

Parser LALR(1)



Parser LALR(1)

	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s11	s13				g9	g7
5				r2			
6	s8	s6				g10	g12
7				r3			
8			r4	r4			
9				r1			
10			r5	r5			
11				r4			
12			r3	r3			
13	s11	s13				g14	g7
14				r5			

(a) LR(1)

	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s8	s6				g9	g7
5				r2			
6	s8	s6				g10	g7
7			r3	r3			
8			r4	r4			
9				r1			
10			r5	r5			

(b) LALR(1)

Parser LALR(1)

- Pode gerar uma tabela com conflitos, onde LR(1) não possuía.

Parser LALR(1)

- Pode gerar uma tabela com conflitos, onde LR(1) não possuía.
 - Na prática, o efeito de redução no uso de memória é bastante desejável.

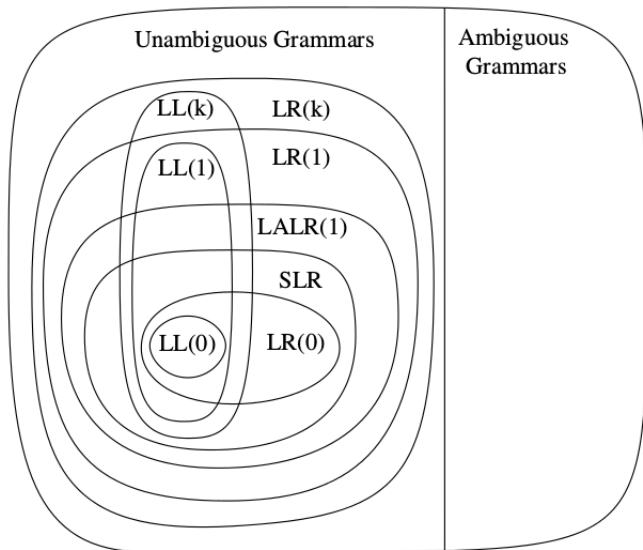
Parser LALR(1)

- Pode gerar uma tabela com conflitos, onde LR(1) não possuía.
 - Na prática, o efeito de redução no uso de memória é bastante desejável.
- A maioria das linguagens de programação é LALR(1).

Parser LALR(1)

- Pode gerar uma tabela com conflitos, onde LR(1) não possuía.
 - Na prática, o efeito de redução no uso de memória é bastante desejável.
- A maioria das linguagens de programação é LALR(1).
- É o tipo mais usado em geradores automáticos de parsers.

Hierarquia das gramáticas



Ambiguidade

$$\begin{aligned} S &\longrightarrow \text{if } E \text{ then } S \text{ else } S \\ S &\longrightarrow \text{if } E \text{ then } S \\ S &\longrightarrow \text{other} \end{aligned}$$

Como seria a parser tree para:

if a then if b then s1 else s2

Ambiguidade

```
S → if E then S else S
S → if E then S
S → other
```

Ambiguidade

$$\begin{aligned} S &\longrightarrow \text{if } E \text{ then } S \text{ else } S \\ S &\longrightarrow \text{if } E \text{ then } S \\ S &\longrightarrow \text{other} \end{aligned}$$

- (1) if a then {if b then s1 else s2}
- (2) if a then {if b then s1} else s2

Ambiguidade

$$\begin{aligned} S &\rightarrow \text{if } E \text{ then } S \text{ else } S \\ S &\rightarrow \text{if } E \text{ then } S \\ S &\rightarrow \text{other} \end{aligned}$$

- (1) if a then {if b then s1 else s2}
- (2) if a then {if b then s1} else s2

Conflito SHIFT-REDUCE!

$\begin{aligned} S &\rightarrow \text{if } E \text{ then } S . && \text{else} \\ S &\rightarrow \text{if } E \text{ then } S . \text{else } S && (any) \end{aligned}$

Eliminando

```

S  → M
S  → U
M  → if E then M else M
M  → other
U  → if E then S
U  → if E then M else U

```

Como seria a parser tree para:

```
if a then if b then s1 else s2
```

Eliminando

- Pode-se usar a gramática ambígua, decidindo os conflitos sempre por SHIFT em casos desse tipo.

Eliminando

- Pode-se usar a gramática ambígua, decidindo os conflitos sempre por SHIFT em casos desse tipo.
- Somente aconselhável em casos bem conhecidos.

Diretivas de precedência

- Nenhuma gramática ambígua é LR(k), para nenhum k.

Diretivas de precedência

- Nenhuma gramática ambígua é LR(k), para nenhum k.
- Podemos usá-las se encontrarmos uma maneira de resolver os conflitos.

Diretivas de precedência

- Nenhuma gramática ambígua é LR(k), para nenhum k.
- Podemos usá-las se encontrarmos uma maneira de resolver os conflitos.
- Relembrando um exemplo anterior...

Diretivas de precedência

1. $E \rightarrow id$
2. $E \rightarrow num$
3. $E \rightarrow E * E$
4. $E \rightarrow E / E$

5. $E \rightarrow E + E$
6. $E \rightarrow E - E$
7. $E \rightarrow (E)$

1. $E \rightarrow E + T$
2. $E \rightarrow E - T$
3. $E \rightarrow T$

4. $T \rightarrow T * F$
5. $T \rightarrow T / F$
6. $T \rightarrow F$

7. $F \rightarrow id$
8. $F \rightarrow num$
9. $F \rightarrow (E)$

Diretivas de precedência

	id	num	+	-	*	/	(	)	\$	<i>E</i>
1	s2	s3					s4			g7
2			r1	r1	r1	r1		r1	r1	
3			r2	r2	r2	r2		r2	r2	
4	s2	s3					s4			g5
5								s6		
6			r7	r7	r7	r7		r7	r7	
7			s8	s10	s12	s14			a	g9
8	s2	s3					s4			
9			s8,r5	s10,r5	s12,r5	s14,r5		r5	r5	
10	s2	s3					s4			g11
11			s8,r6	s10,r6	s12,r6	s14,r6		r6	r6	
12	s2	s3					s4			g13
13			s8,r3	s10,r3	s12,r3	s14,r3		r3	r3	
14	s2	s3					s4			g15
15			s8,r4	s10,r4	s12,r4	s14,r4		r4	r4	

Diretivas de precedência

Vejamos o estado 13:

$E \rightarrow E + E .$	$+$
$E \rightarrow E . + E$	(any)

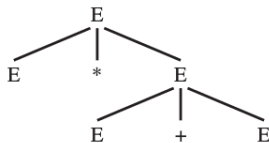
- Pilha:
 - ... E * E (+)
- SHIFT:
 - ... E * E +
- Chegando a:
 - ... E * E + E
- Reduzindo para: ... E * E

- Pilha:
 - ... E * E (+)
- REDUCE:
 - ... E (+)
- Chegando a:
 - ... E +

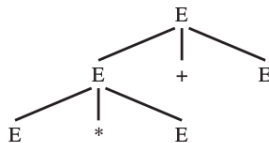
Diretivas de precedência

Vejamos o estado 13:

$E \rightarrow E + E .$	$+$
$E \rightarrow E . + E$	(any)



Shift



Reduce

Qual queremos?

Diretivas de precedência

```
precedence nonassoc EQ, NEQ;  
precedence left PLUS, MINUS;  
precedence left TIMES, DIV;  
precedence right EXP;
```

Diretivas de precedência

```
precedence nonassoc EQ, NEQ;  
precedence left PLUS, MINUS;  
precedence left TIMES, DIV;  
precedence right EXP;
```

Indicaria que:

- + e - têm igual precedência e são associativos à esquerda.

Diretivas de precedência

```
precedence nonassoc EQ, NEQ;  
precedence left PLUS, MINUS;  
precedence left TIMES, DIV;  
precedence right EXP;
```

Indicaria que:

- $+$ e $-$ têm igual precedência e são associativos à esquerda.
- $*$ e $/$ são associativos à esquerda e têm maior precedência que $+$ e $-$.

Diretivas de precedência

```
precedence nonassoc EQ, NEQ;  
precedence left PLUS, MINUS;  
precedence left TIMES, DIV;  
precedence right EXP;
```

Indicaria que:

- $+$ e $-$ têm igual precedência e são associativos à esquerda.
- $*$ e $/$ são associativos à esquerda e têm maior precedência que $+$ e $-$.

Diretivas de precedência

- Podem ajudar.

Diretivas de precedência

- Podem ajudar.
- Não devem ser abusivamente utilizadas.

Diretivas de precedência

- Podem ajudar.
- Não devem ser abusivamente utilizadas.
- Se não consegue explicar ou bolar um uso de precedências que resolva o seu problema, reescreva a gramática.

Análise Sintática



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismailly
ismailybf@ufc.br

Semestre 2025.2

Linguagens de Programação
Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)